



INF-331 Pruebas de Software

Proyecto 2: Entorno CI/CD

Nombre	Javier Torres R.	Fecha	28 Julio de 2022
Sede UTFSM	Casa central	Versión	1.0

Video Proyecto 2: https://drive.google.com/file/d/1CHHkQ1gOL1HWZPJ3nfiD3NqFXC568B_S/view?usp=sharing (Se añade también en AULA junto a este documento)

Repo. Proyecto (Aplicativo Web): <https://github.com/J122016/J122016-Tarea-2-INF331-UTFSM>

En este informe		Página/enlace
1. Descripción de trabajo realizado	_____	2
1.1. Introducción Proyecto 2	_____	2
1.2. Preparativos CI/CD con Jenkins (ambiente + release)	_____	3
1.3. Preparativos y Scripts de prueba de Selenium	_____	4
2. Conclusiones	_____	7

1. Descripción de trabajo realizado

Para la aplicación *Administradora de contactos*, se ocupó la realizada en el proyecto 1 con JavaScript puro acompañado de Bootstrap5. El código se encuentra alojado en el siguiente repositorio GitHub:

<https://github.com/J122016/J122016-Tarea-2-INF331-UTFSM> //favor omitir nombre del repositorio

La aplicación corresponde a una página en donde el usuario puede realizar las siguientes actividades¹:

- Listar contactos (vista por defecto y donde terminan los flujos de otras acciones)
- Agregar contacto
- Editar contacto
- Eliminar contacto

Donde el contacto posee los siguientes campos

- Nombre
- Primer apellido
- Segundo apellido
- Email
- Teléfono

La página web para manejar la persistencia de contactos utiliza el LocalStorage del navegador dedicado al sitio, no se utiliza alguna base de datos dedicada, ya que no es necesario para el objetivo principal del proyecto relacionado a pruebas. Cabe mencionar que en la primera instancia se utilizó Jasmine para elaborar y ejecutar las pruebas referentes a la aplicación.

1.1. Introducción

Para este segundo proyecto, se continua con la base (aplicación) del primero, enfocándose en la implementación de un ambiente de integración continua utilizando Jenkins, junto a realizar un release de esta y la ejecución de un set de pruebas de interfaz utilizando Selenium. En las siguientes secciones se profundiza sobre estas.

¹ Las especificaciones y supuestos de estas acciones están señaladas en el enunciado del Proyecto 1.

1.2. Preparativos CI/CD con Jenkins (ambiente + release)

En este caso se escogió trabajar Jenkins (con su versión estable TLS) en un ambiente local instanciado por Docker 4.10.1 por facilidad de replicación y de uso, junto con no contar con una máquina externa.

Para el uso de Docker se creó un iniciador Docker compose en el cual se configuró el ambiente de Jenkins

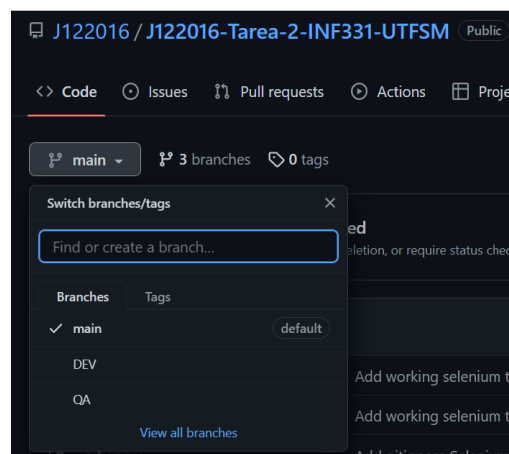
Docker compose para el servidor Jenkins

```
docker-compose.yml X
Env > docker-compose.yml
1  services:
2      jenkins:
3          image: jenkins/jenkins:lts
4          privileged: true
5          user: root
6          ports:
7              - 8080:8080
8              - 50000:50000
9          container_name: jenkins_P2_CICD
10         volumes:
11             - /jenkins_compose/jenkins_configuration:/var/jenkins_home
12             - /var/run/docker.sock:/var/run/docker.sock
```

Posteriormente se ejecutó el compose y se abrió el servidor iniciando así la configuración normal básica de Jenkins descrita en el documento que se encuentra en Aula “Instalación Jenkins (manual corta palo)”.

Para **simular los ambientes**, en el repositorio inicial se crearon 3 ramas:

- **DEV**, donde se desarrolla la aplicación continuamente
- **QA**, donde se enviarán y se revisarán los cambios, comprobándolo con un job de Jenkins configurado para que revise esta rama cuando exista un push a github (modificable)
- **main**, equivalente a producción, la cual pasará código solo aceptado por el Job de la rama anterior generando así un release.



Separación de las ramas en git simulando los ambientes a trabajar

Creación del Job, configuración destacable (idea, no se alcanzó a automatizar con Jenkins):

- Activación de la opción GitHub hook trigger for GITScm polling, esto para que un webhook diera la iniciación del job para realizar un build cuando se encuentre un push dentro de la rama **DEV**.
- Se procede a realizar un merge de la rama **QA** con los nuevos cambios pertenecientes a **DEV**.
- Junto a lo anterior se ejecuta el set de pruebas creado en Selenium (explicado en detalle en sección siguiente) para analizar el comportamiento de la aplicación.
- Si pasa es set de pruebas se realiza un merge en la rama **main** con lo nuevo de **QA** para finalmente realizar un push al entorno principal generándose un release; de lo contrario se etiqueta la versión como fallida y se envía un mail con el log del set de pruebas.

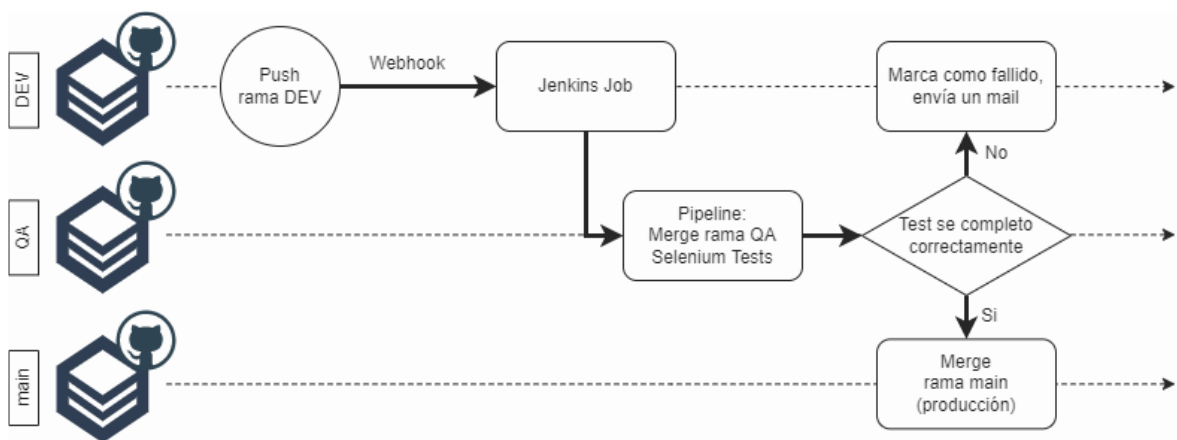
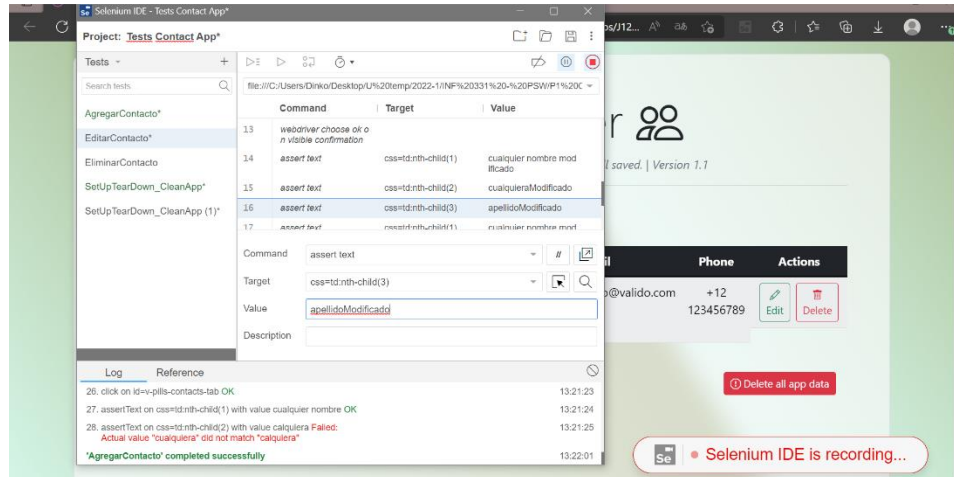


Diagrama de Integración de soluciones para la generación de un release con Jenkins.

1.3. Preparativos y Scripts de prueba de Selenium

Para **Selenium se inicio** instalando el webdriver del navegador específico, junto con añadirlo al path. Además, se instaló el complemento IDE de Selenium para acelerar el proceso de la creación de las pruebas, el cual es detallado a continuación.

Mediante el **IDE se grabó una sesión** donde se realizaron los pasos para las acciones/requerimientos disponibles de la aplicación, así posteriormente se exporto el set de pruebas a lenguaje Python para tener una base rápida.



Proceso de grabación de interacciones usuarias para la generación de una base de pruebas.

Más tarde se procedió a **editar la base producida** agregando:

- Las **comprobaciones de lectura y sincronizaciones de ejecución**, esto último debido a que la automatización ocurre con una velocidad que no permitía que el sitio se renderizase correctamente, esto se solucionó usando `wait` `webdriver` y verificaciones de interactividad de Selenium.
- **Secciones SetUp y TearDown** para limpiar posibles datos al inicio y final de la ejecución, aunque no es estrictamente necesario ya que Selenium genera una sesión nueva en cada ejecución.
- Lógica del main para ejecutar el set de pruebas desarrollado
- **Incorporación de logs** para conocer las pruebas detalladas realizadas y/o errores ocurridos, eso porque algunos de los lanzados por Selenium no eran muy descriptivos.

```
J122016-Tarea-2-INF331-UTFSM > Selenium > SeleniumTestLog
1 2022-07-28 19:05:33,026 | Executing all test, actions flow (CRUD, implicit Read) in normal order...
2 2022-07-28 19:05:33,027 | 0.0/4 - Setup driver...
3 2022-07-28 19:05:38,721 | 0.1/4 - Cleaning old data
4 2022-07-28 19:05:40,557 | -> 'Test' test_setUpTearDownCleanApp completado correctamente!
5 ----
6 2022-07-28 19:05:40,559 | 1.0/4 - Add new contact
7 2022-07-28 19:05:40,560 | - Intentando agregar un contacto con datos invalidos
8 2022-07-28 19:05:43,096 | - Verificando que no se añadio a la tabla 1/2
9 2022-07-28 19:05:43,879 | - Variando otros campos para invalidez
10 2022-07-28 19:05:45,180 | - Verificando que no se añadio a la tabla 2/2
11 2022-07-28 19:05:48,306 | - Agregando un contacto correctamente
12 2022-07-28 19:05:49,164 | - Validando datos del contacto guardado
13 2022-07-28 19:05:49,981 | -> Test test_agregarContacto completado correctamente!
14 ----
15 2022-07-28 19:05:49,982 | 2.0/4 - Edit contact but retracted to avoid overwrite
16 2022-07-28 19:05:49,983 | - Intentando agregar un nuevo contacto con telefono/id ya existente
17 2022-07-28 19:05:51,843 | - Rechazando sobreescritura
18 2022-07-28 19:05:51,887 | - Revisando datos no fueron sobreescritos
19 2022-07-28 19:05:52,867 | -> Test test_editarContactoFallidoSinReemplazar completado correctamente!
20 ----
21 2022-07-28 19:05:52,868 | 2.1/4 - Edit contact, overwrite old
22 2022-07-28 19:05:52,868 | - Editando un nuevo contacto manteniendo telefono/id
23 2022-07-28 19:05:54,393 | - Aceptando sobreescritura
24 2022-07-28 19:05:54,538 | - Revisando datos fueron sobreescritos
25 2022-07-28 19:05:55,108 | -> Test test_editarContactoCorrectamenteSinTel completado correctamente!
26 ----
```

Adición de log, resultados.

El **Detalle del script** de prueba se muestra a continuación siendo un resumen de las pruebas debido a la gran longitud final.

Prueba creación de contacto (extracto)

```
56 def test_agregarContacto(self):
57     #self.driver = app_path(self.driver) #sino prueba en otra ventana la cual hay que cerrar .close()
58     wait = WebDriverWait(self.driver, 30)
59     AddLog("- Intentando agregar un contacto con datos invalidos")
60     contactFormTab = wait.until(expected_conditions.element_to_be_clickable((By.ID, "v-pills-contact-form-tab")))
61     contactFormTab.click()
62     formSubmitButton = wait.until(expected_conditions.element_to_be_clickable((By.NAME, "Save contact")))
63     self.driver.find_element(By.ID, "inputName").clear() #clear()->just in case other test before should not but
64     self.driver.find_element(By.ID, "inputName").click()
65     self.driver.find_element(By.ID, "inputName").send_keys("cualquier nombre")
66     self.driver.find_element(By.ID, "inputPhone").clear()
67     self.driver.find_element(By.ID, "inputPhone").click()
68     self.driver.find_element(By.ID, "inputPhone").send_keys("no valido")
69     self.driver.find_element(By.ID, "inputSurname1").clear()
70     self.driver.find_element(By.ID, "inputSurname1").click()
71     self.driver.find_element(By.ID, "inputSurname1").send_keys("cualquiera")
72     self.driver.find_element(By.ID, "inputSurname2").clear()
73     self.driver.find_element(By.ID, "inputSurname2").click()
74     self.driver.find_element(By.ID, "inputSurname2").send_keys("apellido")
75     self.driver.find_element(By.ID, "inputEmail").clear()
76     self.driver.find_element(By.ID, "inputEmail").click()
77     self.driver.find_element(By.ID, "inputEmail").send_keys("correo@valido.com")
78     self.driver.find_element(By.NAME, "Save contact").click()
79     AddLog("- Verificando que no se añadio a la tabla 1/2")
80     contactTableTab = wait.until(expected_conditions.element_to_be_clickable((By.ID, "v-pills-contacts-tab")))
81     contactTableTab.click()
82     DynamicContactTable = wait.until(expected_conditions.visibility_of(self.driver.find_element(By.ID, "DynamicContactTable")))
83     elements = self.driver.find_elements(By.XPATH, "//div[@id='DynamicContactTable-tbody\']")
84     assert len(elements) == 0
85     self.driver.implicitly_wait(3) #End of minitest1 + animation slow problem , maybe improve splitting in others test
```

Es posible observar claramente la sección donde se limpian los campos, se realiza clic y se escribe en ellos, este extracto la sub sección de la prueba era dado un contacto con datos no válidos, que este no sea añadido a la tabla principal.

Prueba edición de contacto

```
154 def test_editarContactoCorrectamenteSinTel(self):
155     #self.driver = app_path(self.driver)
156     AddLog("- Editando un nuevo contacto manteniendo telefono/id")
157     wait = WebDriverWait(self.driver, 30)
158     self.driver.find_element(By.XPATH, "//tr[@id='+12 123456789']/td[6]/button").click()
159     formSubmitButton = wait.until(expected_conditions.element_to_be_clickable((By.NAME, "Save contact")))
160     self.driver.find_element(By.ID, "inputName").clear()
161     self.driver.find_element(By.ID, "inputName").send_keys("cualquier nombre modificado")
162     self.driver.find_element(By.ID, "inputEmail").clear()
163     self.driver.find_element(By.ID, "inputEmail").send_keys("correoModificado@valido.com")
164     self.driver.find_element(By.ID, "inputSurname1").clear()
165     self.driver.find_element(By.ID, "inputSurname1").send_keys("cualquieraModificado")
166     self.driver.find_element(By.ID, "inputSurname2").clear()
167     self.driver.find_element(By.ID, "inputSurname2").send_keys("apellidoModificado")
168     #editar telefono no es necesario, al editar app autocompleta, add assert
169     formSubmitButton.click()
170     AddLog("- Aceptando sobreescritura")
171     wait.until(expected_conditions.alert_is_present())
172     alert = self.driver.switch_to.alert
173     assert alert.text == "Phone already exist.\nDo you wish overwrite contact?, otherwise cancel."
174     alert.accept()
175     AddLog("- Revisando datos fueron sobreescritos") #cambio a tabla automatico
176     DynamicContactTable = wait.until(expected_conditions.visibility_of(self.driver.find_element(By.ID, "DynamicContactTable")))
177     assert self.driver.find_element(By.XPATH, "//tr[@id='+12 123456789']/td[1]").text == "cualquier nombre modificado"
178     assert self.driver.find_element(By.XPATH, "//tr[@id='+12 123456789']/td[2]").text == "cualquieraModificado"
179     assert self.driver.find_element(By.XPATH, "//tr[@id='+12 123456789']/td[3]").text == "apellidoModificado"
180     assert self.driver.find_element(By.XPATH, "//tr[@id='+12 123456789']/td[4]").text == "correoModificado@valido.com"
181     AddLog("-> Test test_editarContactoCorrectamenteSinTel completado correctamente!\n---")
```



En esta prueba, se omitieron algunas instrucciones debido a que la aplicación se debe encargar de aquellas, por ejemplo, el paso del formulario a la tabla general, o el escribir nuevamente el número de teléfono, ya que como se está editando, los datos anteriores se ingresan automáticamente. Otro aspecto importante a mencionar es la simulación de la aceptación usuaria de la alerta y la posterior comprobación de los datos para finalizar la prueba.

Junto a este archivo y en el repositorio se encuentra el archivo `test_contactAppCRUDTests.py`, el cual se pueden revisar todo el set de pruebas generado y realizado, para su ejecución es necesario el Web driver de selenium para su navegador (Edge por defecto) y Python.

2. Conclusiones

Finalmente, al desarrollar este trabajo se comprendió mejor la separación CI/CD y la importancia de tener ambientes separados de desarrollo, pruebas y producción.

También se comprobó que los orquestadores que facilitan estas tareas como por ejemplo Jenkins, el cual puede ejecutarse continuamente en un servidor externo para realizar cíclicamente labores de integración y posterior despliegue. Sin embargo, un detalle no menor encontrado fue la pequeña cantidad de documentación en línea en comparación a otras herramientas de desarrollo.

Por otra parte, con Selenium se pudo simular las interacciones exactas del usuario similar a una prueba de caja negra, a diferencia de otras herramientas que permiten probar el código como caja blanca utilizando las funciones.

A la par que se desarrolló esta parte del proyecto se evidenció lo exacto y costoso en términos de tiempo que es configurar estos ambientes y herramientas, debido a todos los detalles involucrados, sin embargo, al finalizar se observó lo útil para acelerar las siguientes iteraciones de desarrollo ahorrándose el tiempo en tareas manuales, aun así, están siguen siendo necesarias en el desarrollo posterior modular del aplicativo por parte de cada desarrollador.